Maximum flow

Part VI Graph Algorithms

Scope

Residual networks, cut certificates, and valid heights prove maximum flow. Complete traces reconstruct augmentation, matching, and relabel-to-front.

Prerequisites

Directed graphs, BFS, and summation notation. This chapter develops conservation, residual networks, and cuts.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Define flow networks and net flow value.
- Use residual edges to revise Ford–Fulkerson choices.
- Prove max-flow min-cut and connect bipartite matching.
- 26.1 Flow networks
- 26.2 The Ford-Fulkerson method
- 26.3 Maximum bipartite matching
- 26.4 Push-relabel algorithms
- 26.5 The relabel-to-front algorithm

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.1 Flow networks

A capacity network has finite V, distinct source s and sink t, and capacities $c(u, v) \geq 0$. There are no self-loops; nonedges have capacity 0, and same-direction parallel capacities may be combined. Write $n = |V| \geq 2$ and m for the number of positive-capacity directed edges. Isolated vertices and vertices outside all s-to-t paths are allowed.

Use antisymmetric net flow $f(u, v) = -f(v, u)$ with $f(u, v) \leq c(u, v)$. Its full range is $-c(v, u) \leq f(u, v) \leq c(u, v)$. Internal $u \neq s, t$ obeys $\Sigma_v f(u, v) = 0$. Flow value $|f| = \Sigma_v f(s, v)$ is net source outflow, not a sum of absolute edge flows.

Requiring nonnegativity for all ordered net-flow pairs would force every value to 0 by antisymmetry. Capacities may be real. First count exact arithmetic operations; integer and rational bit costs are separate.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Nonnegative arc flow differs from net flow

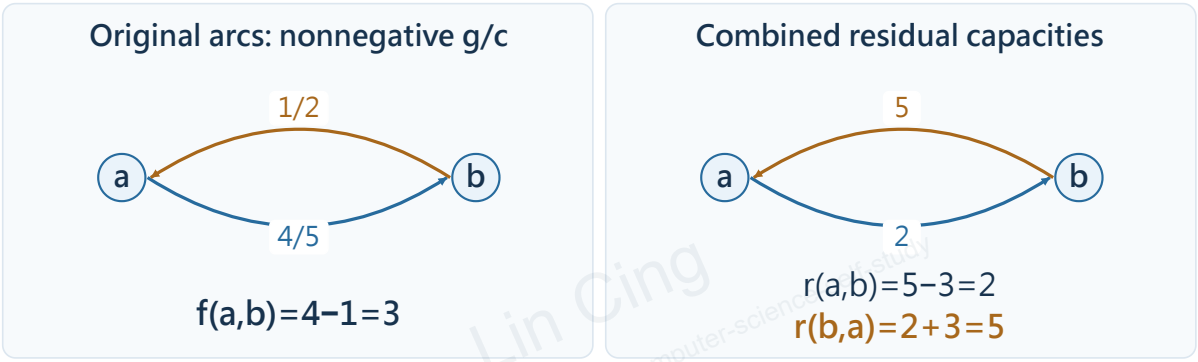
An alternative stores $0 \leq g(u, v) \leq c(u, v)$ on original arcs, without antisymmetry. Define $f(u, v) = g(u, v) - g(v, u)$, preserving conservation and net value. Opposite arc flows can cancel $\min(g(u, v), g(v, u))$ without changing any vertex's balance.

Conversely, for net f define $g(u, v) = \max(f(u, v), 0)$. Capacity bounds give $g \leq c$, and $g(u, v) - g(v, u) = f(u, v)$. Thus the models have the same maximum net value.

With $c(a, b) = 5$, $c(b, a) = 2$ and $g(a, b) = 4$, $g(b, a) = 1$, net values are 3 and -3. Canceling 1 gives arc flows 3 and 0. Forward residual capacity is $5 - 3 = 2$; reverse residual is $2 - (-3) = 5$, combining withdrawal of 3 with 2 units of original reverse capacity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Original reverse capacity and cancellation are distinct



Reverse capacity 5 = original reverse capacity 2 + cancellation of net flow 3

For this pair of arcs, capacities are 5 and 2, original arc flows are 4 and 1, and net flow is 3. Residual b→a allows 5: undo 3 units of net flow, then use 2 units of original reverse capacity. **A residual arc need not be an original arc; reverse residual capacity need not consist only of cancellation.**

Internal cancellation gives the cut upper bound

A cut(S, T) partitions V with s∈S, t∈T. Its capacity c(S, T)=Σ_(u∈S, v∈T)c(u, v) counts only S-to-T arcs. Sum net outflows over S: source contributes |f| and other vertices contribute0. Internal antisymmetric pairs cancel, giving

$$|f| = \sum_{u \in S, v \in T} f(u,v) \leq \sum_{u \in S, v \in T} c(u,v) = c(S,T).$$

Count net crossing flow, not merely positive contributions. A feasible flow certifies a lower bound on the optimum; any cut certifies an upper bound. Equality is a checkable optimality certificate.

Flow and cut certificates on the same network

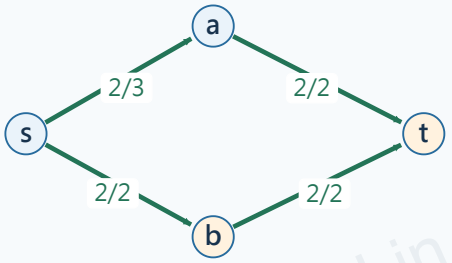
Capacities are s→a=3, a→t=2, s→b=2, b→t=2. Send2 along each s-a-t and s-b-t. Conservation and capacity bounds hold, with value 4.

For S={s, a}, T={b, t}, crossing capacities s→b:2 and a→t:2 sum to 4. Thus the maximum is both at least and at most4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Flow and cut certificates on the same network (continued)

Feasible flow: original arcs show f/c



S={s,a}; T={b,t}

s → b : 2

a → t : 2

c(S,T) = 2+2 = 4

|f| = 2+2 = 4

Flow gives a lower bound; cut gives an upper bound; equality proves optimality

Source outcapacity or sink incapacity alone does not establish internal connectivity. The full network, feasible flow, and equal cut establish optimality here.

Multiple sources and sinks with finite added capacities

For disjoint supply set A and demand set B without individual bounds, add $s' \to a$ for each $a \in A$ and $b \to t'$ for each $b \in B$. Use total original capacity C for added capacities instead of an implemented ∞; no useful transport exceeds that total. If individual endpoint limits exist, use those actual limits.

Attach original endpoint net supplies to the new arcs to obtain conservation. Removing those arcs recovers the original supplies and demands at unchanged total value. This establishes both directions of the reduction.

To eliminate antiparallel original arcs in a nonnegative-arc model, replace $u \to v$ of capacity c by $u \to x \to v$ with both capacities c. Conservation at x forces equal segment flow, giving an exact correspondence. Setting both capacities to $c/2$ would change the problem.

26.2 The Ford-Fulkerson method

Residual capacity is $c_f(u, v) = c(u, v) - f(u, v) \geq 0$. The residual graph contains exactly positive-residual arcs. Neighbor candidates include original reverse directions even when their original capacity is 0.

For a simple residual s-to-t path p, let $\delta = \min_{(uv \in p)} c_f(u, v) > 0$. Update $f(u, v) \mathrel{+}= \delta$ and $f(v, u) \mathrel{-}= \delta$ together. Forward capacity is respected; reverse decrease cannot violate its upper bound, and its lower bound is equivalent to the forward upper bound. Path-internal balance changes cancel. Source net outflow increases by δ, preserving feasibility and strictly increasing value.

```
f=zero net flow
while a residual s-t path p exists:
    delta=min(residual(u, v) for (u, v) in p)
    for (u, v) in p:
        f(u, v)+=delta; f(v, u)-=delta
return f
```

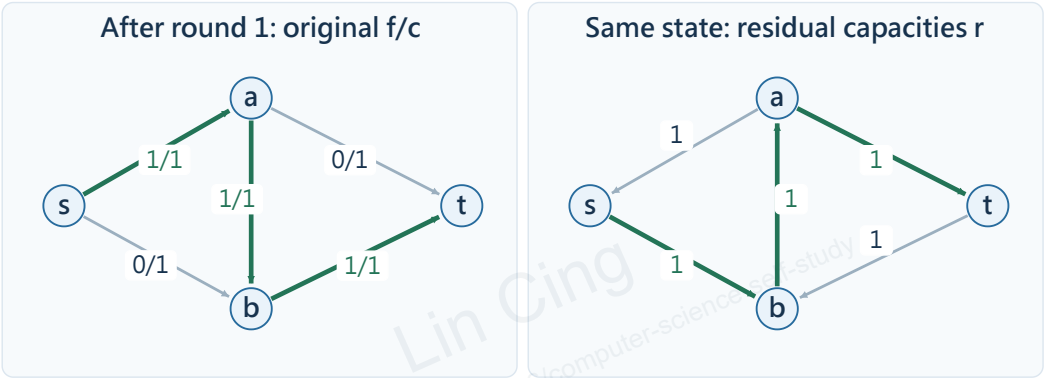
Two augmentations that withdraw an earlier choice

All original capacities s→a, s→b, a→b, a→t, b→t are 1.

Stage	Path and δ	(f_sa, f_sb, f_ab, f_at, f_bt)	Value
Initial	none	0,0,0,0,0	0
First	s→a→b→t; 1	1,0,1,0,1	1
Second	s→b→a→t; 1	1,1,0,1,1	2

Residual b→a cancels old a→b flow from 1 to 0. The final flow sends 1 on s-a-t and 1 on s-b-t. Restricting search to original directions would incorrectly stop after the first augmentation and return 1.

Two augmentations that withdraw an earlier choice (continued)

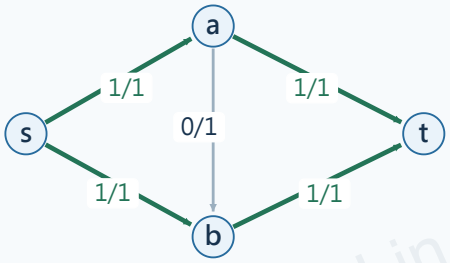


Green residual path s→b→a→t has bottleneck 1; b→a can undo old flow

When storing residual capacities directly, equivalent updates are r(u, v)−=δ, r(v, u)+=δ. Decreasing only the forward entry loses withdrawal capability.

After cancellation, check original arcs and optimality

After round 2: feasible value 2



Check every original arc

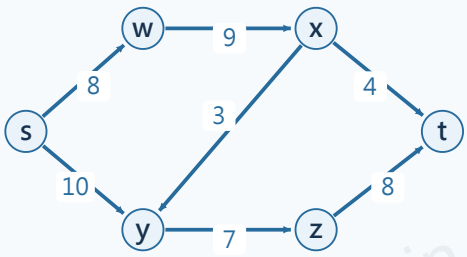
s→a	1 → 1
s→b	0 → 1
a→b	1 → 0
a→t	0 → 1
b→t	1 → 1

a→b falls from 1 to 0; both source arcs are saturated; cut {s} has capacity 2

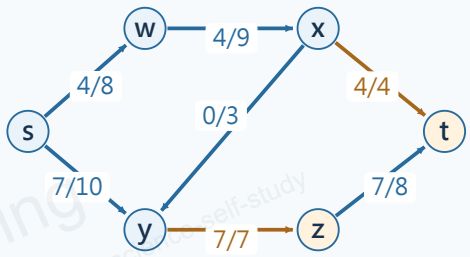
The second augmentation updates s→b, b→a, a→t. Using b→a decreases net f(a,b), without removing its original capacity. Vertices a and b each receive and send 1. Flow value 2 equals cut {s} capacity 2. Residual a→b still has capacity 1, but s cannot reach a, so it permits no further augmentation.

Six vertices: how do we reach the optimum?

Six vertices: original capacities c



After four rounds: original f/c



S={s,w,x,y}: cut arcs x→t:4 and y→z:7 total 11, equal to the flow

The original graph has x→y:3, but no y→x. The right panel shows original f/c after four augmentations, not residual capacities. Use vector order (sw,wx,xy,xt,sy,yz,zt) throughout the trace.

First send x→y, then undo it along y→x

Round	Path; residual capacities before augmentation	δ	Original flows afterward
Start	none	none	(0,0,0,0,0,0,0)
1	s-w-x-y-z-t; (8,9,3,7,8)	3	(3,3,3,0,0,3,3)
2	s-y-x-t; (10,3,4)	3	(3,3,0,3,3,3,3)

Round 1 creates residual y→x:3. Round 2 uses it, reducing original f(x,y) from 3 to 0. It does not add an original reverse arc. Now x receives 3 through wx and sends 3 through xt; y receives 3 through sy and sends 3 through yz. Source net outflow is 3+3=6.

Finish the trace and verify the cut

Vectors remain ordered (sw,wx,xy,xt,sy,yz,zt).

Round	Path; residual capacities before augmentation	δ	Original flows afterward
3	s-y-z-t; (7,4,5)	4	(3,3,0,3,7,7,7)
4	s-w-x-t; (5,6,1)	1	(4,4,0,4,7,7,7)

Value rises 6→10→11. Vertices w,x each receive and send 4; y,z each receive and send 7. Residual reachability gives S={s,w,x,y}. The only original S→T arcs are x→t:4 and y→z:7. **Feasible value 11 and cut capacity 11 prove optimality.** Source outcapacity 8+10=18 alone cannot determine the answer.

Three equivalent max-flow min-cut conditions

For feasible f, these are equivalent:(i) f is maximum;(ii) no residual s-to-t path exists;(iii) some cut satisfies $|f|=c(S, T)$.

(i)⇒(ii): an existing path would augment to a larger feasible value.

(ii)⇒(iii): let S be residual-reachable from s and T its complement. Since $t \notin S$, this is a cut. Every $u \in S$, $v \in T$ has $c_f(u, v)=0$, or v would be reachable. Thus $f(u, v)=c(u, v)$, and the net cut identity gives equality.

(iii)⇒(i): every feasible g has $|g| \leq c(S, T)=|f|$.

Neither integer capacities nor a specific path rule is required. Integrality governs termination and cost, not this equivalence.

Integrality and pseudopolynomial time

With integer capacities and initial zero flow, residuals and δ remain integers. Each augmentation increases value by at least 1. A finite cut bounds the value, so the algorithm terminates with an integral maximum net flow after at most optimum F^* augmentations.

Use adjacency lists, $O(n+m)$ initial allocation, and timestamped visited marks that initialize only touched vertices. Each newly seen nonsource vertex corresponds to a scanned edge, so one path search costs $O(m+1)$. Total time is $O(n+m+(m+1)F^*)$. Since F^* can be exponential in binary input length, this is pseudopolynomial.

Rational capacities can be scaled by the product D of denominators. Its bit length is at most the sum of denominator bit lengths, but its numeric value can be enormous, so augmentation count may still be huge. Irrational capacities do not ensure improvement by at least 1; this argument does not prove termination for arbitrary paths. BFS supplies a capacity-independent bound below.

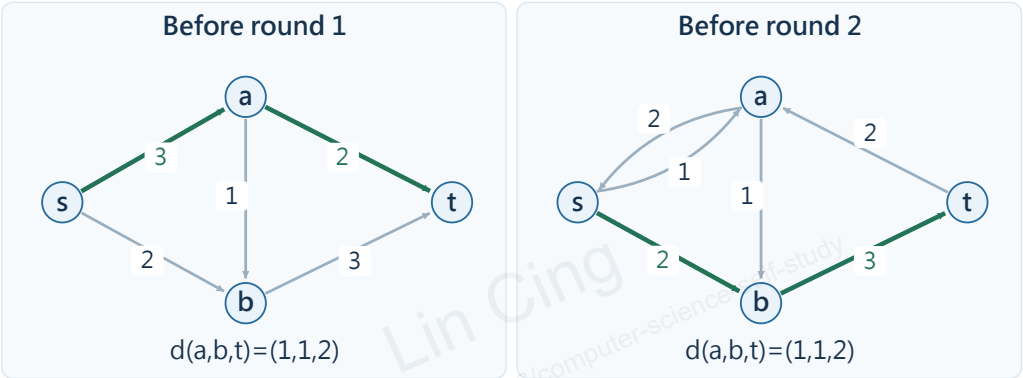
Edmonds–Karp minimizes residual edge count

Use BFS to choose a fewest-edge residual path, then augment through parents. Let d_f(v) be shortest edge count from s, or∞. For s→a=3, s→b=2, a→b=1, a→t=2, b→t=3 and neighbor a before b:

Before augmentation	d(a), d(b), d(t)	Path and δ	Flow tuple afterward
Round 1	1,1,2	s→a→t; 2	2,0,0,2,0
Round 2	1,1,2	s→b→t; 2	2,2,0,2,2
Round 3	1,2,3	s→a→b→t; 1	3,2,1,2,3
Final check	∞,∞,∞	no path	value 5; cut{s} has capacity 5

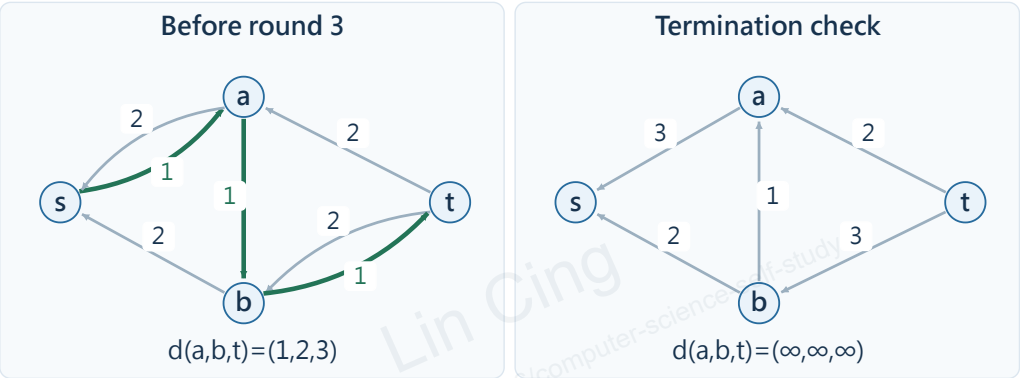
The tuple order is(f_sa, f_sb, f_ab, f_at, f_bt).

Edmonds–Karp minimizes residual edge count (continued)



All positive residual arcs shown; green marks the BFS path; bottlenecks 2, 2

Round 3 and termination: read the cut from residual reachability



Round 3 has δ=1; finally s has no outgoing arc: flow 5 equals cut {s} capacity 5

Path s→a→b→t has bottleneck min(1,1,1)=1, increasing value 4→5. The final residual arcs are a→s:3, b→s:2, b→a:1, t→a:2, t→b:3. Only s is reachable from s, so cut {s} has original capacity 3+2=5, equal to the flow. Distance ∞ means unreachable, not infinite capacity.

Residual BFS distances never decrease

Write d before and d′ after augmentation. Suppose $d'(v)<d(v)$, choosing such a v with smallest $d'(v)$. Its predecessor u on a new shortest path has $d'(u)=d'(v)-1$ and cannot be a smaller decrease counterexample, so $d(u)\le d'(u)$.

If uv was already residual, $d(v)\le d(u)+1\le d'(v)$, a contradiction. Therefore uv is a newly created reverse edge, and vu lies on the last BFS augmentation path. A prefix of that shortest path is shortest, so $d(u)=d(v)+1$.

Combining gives $d(v)=d(u)-1\le d'(u)-1=d'(v)-2$, contradicting $d'(v)<d(v)$. The old-unreachable/new-reachable∞ case also leads to finite old predecessors and hence old reachability, a contradiction. Every distance is nondecreasing.

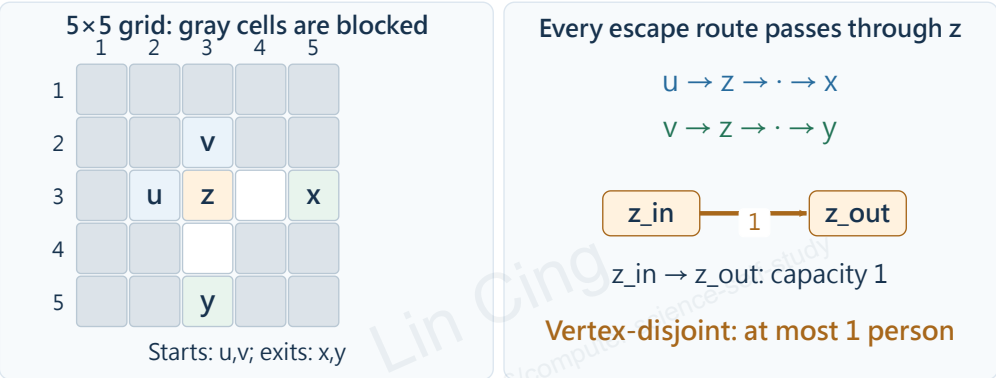
A critical edge's tail rises by at least two before reuse

Every augmentation saturates at least one critical uv at bottleneck δ, with $d(v)=d(u)+1$. For that direction to reappear, a later augmentation must traverse vu. Then $d'(u)=d'(v)+1 \geq d(v)+1=d(u)+2$, using distance monotonicity.

Thus between successive critical uses of one direction, its tail distance increases by at least 2. On an augmenting simple path the tail distance is at most $n-2$, so each direction is critical $O(n)$ times. At most$2m$ candidate directions give $O(nm)$ augmentations.

Each BFS and update takes $O(m+1)$, giving $O(n+m+nm^2)$ for $m \geq 1$, usually written $O(nm^2)$ after initialization. For $m=0$, return zero flow in $O(n)$. Common-denominator scaling of rational capacities keeps all value bit lengths polynomial in input length, and the number of rounds is independent of their magnitude. Thus the algorithm is also polynomial in bit cost; $O(nm^2)$ first counts arithmetic operations.

Escape routes: edge-disjoint does not mean vertex-disjoint



Edge capacities alone give 2; vertex capacities correctly limit escape to 1

Coordinates are (row,column). Open cells are {(3,2),(2,3),(3,3),(3,4),(3,5),(4,3),(5,3)}; all other cells are blocked. Starts are u=(3,2), v=(2,3); boundary exits are x=(3,5), y=(5,3). Unit edge capacities allow two routes sharing z=(3,3). **If no cell may be shared, flow value 2 in that model does not prove both people can escape.**

Split every cell into an entry and an exit

For a $k \times k$ grid, let q be the open-cell count, p the number of distinct occupied starts, and b the open-boundary count. To forbid all shared cells:

Constraint	Network representation
At most one person per cell	$v_{in} \to v_{out}$, capacity 1
Move between adjacent open cells	$u_{out} \to v_{in}$ and $v_{out} \to u_{in}$, each capacity 1
One person per start	source $\to$ start_in, capacity 1
Leave through an open boundary	boundary_out $\to$ sink, capacity 1

Omit blocked cells. A boundary start still traverses its internal edge. The network has $N = 2q + 2$ vertices and $M \le q + 4k(k-1) + p + b$ arcs. For a full open grid, $b = 4k - 4$ requires $k \ge 2$; $b = 1$ when $k = 1$.

Why flow p is equivalent to everyone's escape

Given p vertex-disjoint escape routes, send one unit along each corresponding network path. Conversely, integer capacities admit an integral maximum flow. Decompose it into source–sink paths and cycles; remove cycles without changing value. Value p saturates all p source arcs. Every internal capacity 1 prevents two paths from using the same cell. Mapping back gives one escape route per person. This proves both directions.

Every route in the example passes through z. Edge $z_{in} \to z_{out}$ forms a capacity-1 cut, so at most one person escapes; route $u \to z \to (3,4) \to x$ attains 1. The edge-only model admits two edge-disjoint routes and has value 2. **The answers differ because the constraints differ.**

26.3 Maximum bipartite matching

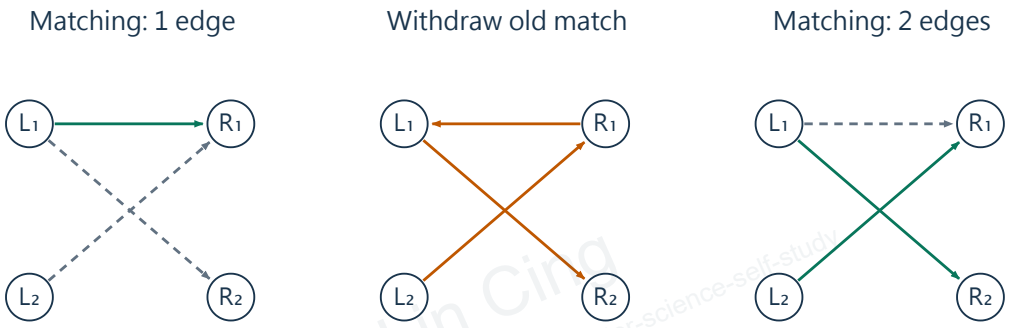
For bipartition $L \cup R$, a matching has no shared endpoints. Build unit arcs $s \to L$, original edges $l \to r$, and $R \to t$.

A matching M sends 1 along each s-l-r-t; disjoint endpoints ensure capacities, and value is $|M|$. Conversely, an integral flow gives each left vertex at most one unit total incoming, so conservation and integrality allow at most one original matching arc with flow 1. The right-side argument is symmetric. Selected unit-flow original arcs form a matching of size equal to the flow value.

Integer augmentation guarantees existence of an integral maximum flow, so maximum matching size equals maximum flow value. It does not make every optimal flow integral: in $K_{2,2}$, put1/2 on all four left-to-right arcs and 1 on source/sink arcs. This optimal value 2 flow does not make all four positive arcs a matching.

Maximum size is at most $\min(|L|,|R|)$, and each augmentation adds 1. With $m' = |E|+|L|+|R|$, direct Ford–Fulkerson costs $O(|V|+m'+m'\min(|L|,|R|))$. Isolated vertices prevent assuming $m' = \Theta(|E|)$ unconditionally.

The actual edges flipped by an alternating path



$L_2 \to R_1 \to L_1 \to R_2$: unmatched, matched backward, unmatched

Green: matching. Gray dashed: unselected original edges. Orange: residual alternating path.

Edges are L_1R_1, L_1R_2, L_2R_1; initially match L_1R_1. Augment along $s \to L_2 \to R_1 \to L_1 \to R_2 \to t$, replacing L_1R_1 by L_2R_1 and L_1R_2. Internal vertices keep one matching edge; both free endpoints become matched, raising size $1 \to 2$.

A constructive proof of Kőnig's equality

For a maximum matching, let Z be residual-reachable from s, and $C=(L \setminus Z) \cup (R \cap Z)$. Consider original lr with $l \in Z$, $r \notin Z$. If unmatched, its forward residual arc reaches r. If matched, the source-to-l arc is saturated; the last arc reaching l must come backward from its matched r, also making r reachable. Both contradict $r \notin Z$. Thus C covers every original edge.

Both endpoints of a matched edge are either reachable or unreachable together: a reachable right endpoint reaches its left mate backward, and a reachable matched left endpoint must have been entered from its right mate. Therefore C contains exactly one endpoint of every matched edge. Unmatched left vertices are source-reachable and outside C. Unmatched right vertices cannot be reachable, or an augmentation would exist, and also lie outside C. Hence $|C|=|M|$.

Any vertex cover must choose a distinct endpoint for every matching edge. Minimum bipartite cover size therefore equals maximum matching size. General graphs need not satisfy this: a triangle has minimum cover2 and maximum matching1.

Sufficiency of Hall's condition

A matching covering L requires $|N(A)| \geq |A|$ for every $A \subseteq L$, because distinct left vertices need distinct neighbors.

If a maximum matching leaves some left vertex unmatched, search alternating paths from unmatched left vertices. Let A, B be reached left and right sets. The preceding forward/backward argument gives $B=N(A)$. Every B vertex is matched, or the search yields an augmentation. Its matched left mate lies in A, which additionally contains an unmatched starting vertex. Hence $|A|>|B|=|N(A)|$, violating Hall's condition.

The condition is sufficient as well as necessary. If L is empty, the empty matching already covers it. Covering all of L is perfect on both sides only when the two side sizes also agree.

26.4 Push-relabel algorithms

A preflow retains antisymmetry and capacity bounds but allows excess $e(u)=\Sigma_v f(v, u)\geq 0$ for $u\neq s$. Active means $u\neq s$, t and $e(u)>0$. The sink accumulates excess but is never discharged.

Valid heights satisfy $h(s)=n$, $h(t)=0$ and $h(u)\leq h(v)+1$ on every residual uv. Initialize other heights and all f, e to 0, then saturate source arcs:$f(s, v)=c(s, v)$, $f(v, s)=-c(s, v)$, $e(v)+=c(s, v)$, $e(s)-=c(s, v)$.

Nonsource excess is nonnegative. No positive residual arc leaves s; other residual arcs start at height0 and satisfy validity. Heights are valid labels, not necessarily current BFS distances.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete push and relabel preconditions

```
push(u, v): # u active, residual>0, h[u]==h[v]+1
    d=min(e[u], c(u, v)-f(u, v))
    f(u, v)+=d; f(v, u)-=d; e[u]-=d; e[v]+=d
relabel(u): # u active, no admissible outgoing edge
    h[u]=1+min(h[v] for v with residual(u, v)>0)
```

Admissible means positive residual and a height drop of exactly 1. A push is saturating if residual becomes 0; otherwise it is nonsaturating and empties e(u). Count simultaneous saturation and emptying as saturating.

An active vertex must have a positive-residual outgoing arc. Otherwise every $f(u, v)=c(u, v)\geq 0$, giving $e(u)=-\Sigma f(u, v)\leq 0$, a contradiction. Thus relabel minimizes a nonempty set. With no admissible arc, integer validity implies $h(u)\leq$every residual neighbor height, so relabel raises h(u) by at least 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Operations preserve preflows and valid heights

Push keeps $e(u)$ nonnegative and only increases other nonsource excess. Capacity and antisymmetry follow as for augmentation. Heights remain unchanged. The only possibly new reverse vu has $h(v)=h(u)-1$, so it remains valid but is not admissible.

Relabel leaves f, e fixed. The new outgoing bound follows from $1+$the minimum neighbor height. Incoming wu remains valid because $h(w)\leq \text{old_}h(u)+1\leq \text{new_}h(u)$, and cannot be admissible. At least one outgoing arc attaining the minimum becomes admissible.

The admissible network is a DAG: a k-edge cycle would imply $h=h-k$. Push creates no admissible edge, and relabel can create only its own outgoing admissible edges. These facts support the later list invariant.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Active vertices reach the source, bounding their heights

Let R be residual-reachable from active u, and suppose $s\notin R$. All R vertices have nonnegative excess and u positive, so $\Sigma_{(v\in R)}e(v)>0$. No positive-residual arc leaves R, so $f(v, w)=c(v, w)\geq 0$ across its outgoing cut. Internal antisymmetric terms cancel, giving

$$\Sigma_{(v\in R)}e(v)=-\Sigma_{(v\in R, w\notin R)}f(v, w)\leq 0,$$

a contradiction. Thus u reaches s along a simple residual path of at most $n-1$ edges. Summing valid-label inequalities gives $h(u)\leq h(s)+n-1=2n-1$. Relabeling leaves u active, so its new label has the same bound; at other times its height is unchanged. Each internal vertex is relabeled at most$2n-1$ times, for $O(n^2)$ total.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Correctness at termination still needs a termination bound

If all internal excesses vanish, the preflow is a feasible flow. A residual s-to-t simple path of $k \leq n-1$ edges would give $n=h(s) \leq h(t)+k=k \leq n-1$, a contradiction. Therefore no augmenting path exists, and max-flow min-cut proves optimality.

Sink excess equals the flow value. Some initial excess can return to the source instead of reaching the sink.

Correctness on termination alone does not prove termination. Bound relabels, saturating pushes, and nonsaturating pushes separately to obtain a finite capacity-independent count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

There are O(nm) saturating pushes

Fix direction uv. At its first saturation, $h(u)=h(v)+1$ and its residual becomes 0. Another positive uv push requires a reverse vu push restoring residual. At that step $h(v)=h(u)+1$; height monotonicity makes v at least 2 higher than at the prior forward saturation.

At the next uv saturation, u must again be one higher than v, and therefore at least 2 higher than its own previous saturation height. Heights are bounded by $2n-1$, so each direction saturates $O(n)$ times. At most$2m$ directions give $O(nm)$ total.

The bound uses height differences and monotonicity, not push quantity, and remains valid for fractional capacities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Potential accounting for nonsaturating pushes

Let $\Phi=\Sigma_{\text{(active u)}}h(u)$. Initially it is 0 and always nonnegative. Relabel increases it exactly by its height increase. Across all vertices, total height increase is at most$(n-2)(2n-1)=O(n^2)$.

A saturating push can activate only its destination, adding at most$2n-1$ to Φ. The $O(nm)$ such pushes contribute $O(n^2m)$ total positive increase. A nonsaturating push empties u. A newly activated v adds $h(v)=h(u)-1$; an already-active v or terminal adds nothing. Thus Φ falls by at least 1.

Nonnegativity bounds these decreases by all positive increases, giving $O(n^2+n^2m)$, or $O(n^2m)$ when $m\geq 1$. Combining all basic operations gives $O(n^2m)$. With $m=0$, initialization leaves no active vertex and terminates directly.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.5 The relabel-to-front algorithm

The linked list L always contains **all** vertices except s, t, including zero-excess vertices. It is not an active-only list. Fix $N(u)=\{v:c(u, v)>0 \text{ or } c(v, u)>0\}$, including reverse candidates. Preserve current across discharges:

```
discharge(u):
    while e[u]>0:
        v=current[u]
        if v==NIL:
            relabel(u); current[u]=head(N[u])
        elif residual(u, v)>0 and h[u]==h[v]+1:
            push(u, v) # do not advance current here
        else: current[u]=next-neighbor(v)
```

A saturated edge is skipped only on the next loop iteration. If push empties u, return immediately with current still pointing to that neighbor. A different implementation that advances after every push cannot share this trace unchanged.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why skipped current arcs need no immediate revisit

Fix an interval with unchanged h(u). If uv is skipped for zero residual, only a reverse vu push can restore it. That requires h(v)=h(u)+1, so restored uv points uphill and remains inadmissible. If uv is residual but inadmissible, valid integer heights imply $h(u) \leq h(v)$. Vertex v can only rise, never creating a one-level descent from fixed u.

Equivalently, push creates no admissible edges, and a neighbor's relabel leaves no admissible incoming edge. Only u's own relabel can create new admissible outgoing arcs; that is exactly when current resets.

Reaching NIL therefore means all skipped arcs are still unusable and relabel is legal. Previously inspecting an edge alone would not justify ignoring later state changes.

Read next from the vertex's new position

```
initialize_preflow()
L=all vertices except s, t, in a fixed order
current[u]=head(N[u]) for u in L
u=head(L)
while u!=NIL:
    old=h[u]; discharge(u)
    if h[u]>old: move u to front of L
    u=next(u) # after any move
```

Move only after the whole discharge, not after each internal relabel. After moving an interior u to the head, its next is the old head, so earlier vertices that may have gained excess are revisited. Saving old next would incorrectly skip them.

Two branches of the list invariant

At every outer-loop test:(1) L is a topological order of the admissible network restricted to nonterminals;(2) all vertices before u have zero excess. Initially nonsource heights are 0, so no admissible arc connects list vertices. Any L works, and u begins at its head.

If discharge does not relabel, push creates no admissible edge, so topological order remains. It sends excess only to later list vertices or terminals, preserving the earlier zero-excess prefix. The discharged u is empty, restoring the prefix condition at next(u).

If discharge relabels, the last relabel leaves u without admissible incoming edges, and subsequent pushes create none. Moving u to the head preserves topological order. Only empty u precedes the new next, and other relative positions stay unchanged.

When u=NIL, the entire list is a zero-excess prefix. Conservation and the generic correctness theorem give maximum flow. This proof does not transfer directly to an active-only scheduling list.

Five-vertex initialization and the complete discharge of a

Capacities are s→a=4, s→b=3, s→c=2, a→b=2, a→t=1, b→c=2, b→t=4, c→t=5. Fix N(a)=[b, t, s], N(b)=[c, t, a, s], N(c)=[t, b, s], L=[a, b, c].

Set h(s)=5, other heights0, and saturate source arcs. Record each push/relabel immediately after it finishes, before subsequent neighbor scanning.

Action	e(a, b, c, t)	h(a, b, c)	current(a, b, c)
Initialize	4,3,2,0	0,0,0	b, c, t
a scans b, t, s to NIL; relabel	4,3,2,0	1,0,0	b, c, t
Push a→b by 2	2,5,2,0	1,0,0	b, c, t
Skip b to t; push a→t by 1	1,5,2,1	1,0,0	t, c, t
Scan t, s to NIL; relabel again	1,5,2,1	6,0,0	b, c, t
Skip b, t to s; return 1 along a→s	0,5,2,1	6,0,0	s, c, t

Five-vertex initialization and the complete discharge of a (continued)

Height of a rises from 0 to 6. Move it to the front after discharge; it was already the head, so L remains unchanged. Next process b.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

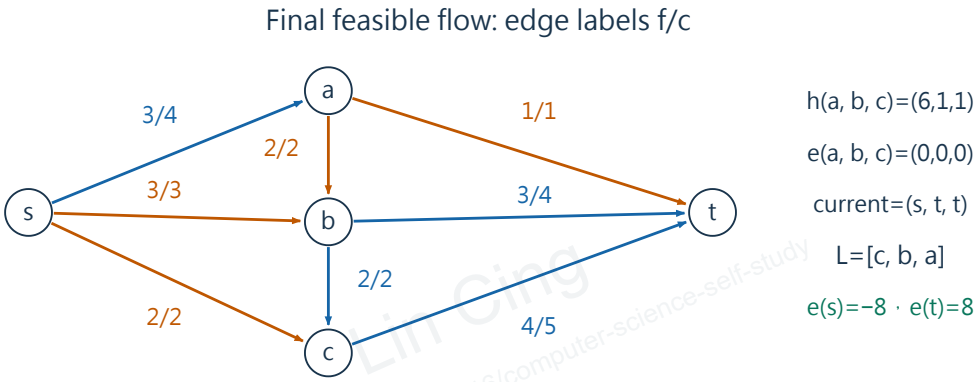
Discharging b, c and the exact move times

Action	e(a, b, c, t)	h(a, b, c)	current(a, b, c)	L
b scans c, t, a, s to NIL; relabel	0,5,2,1	6,1,0	s, c, t	a, b, c
Push b→c by 2	0,3,4,1	6,1,0	s, c, t	a, b, c
Skip c to t; push b→t by 3	0,0,4,4	6,1,0	s, t, t	a, b, c
Move b to front after discharge	0,0,4,4	6,1,0	s, t, t	b, a, c
Revisit a; excess0, no operation	0,0,4,4	6,1,0	s, t, t	b, a, c
c scans t, b, s to NIL; relabel	0,0,4,4	6,1,1	s, t, t	b, a, c
Push c→t by 4; move c after discharge	0,0,0,8	6,1,1	s, t, t	c, b, a

The new next is b, then a, both empty, and finally NIL. Sink excess stays 1 after b→c and becomes 4 only after b→t. Final source net outflow 3+3+2=8 equals sink inflow1+3+4=8.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Independently verify the final flow by its cut



Vertex a receives 3 and sends 2+1; b receives 3+2 and sends 2+3; c receives 2+2 and sends 4. Residual reachability gives S={s,a}; cut capacity 3+2+2+1=8 equals feasible value 8.

One of the initial 9 units returns from a to s, leaving net value 8. No flow is lost. The cut certificate is independent of heights and scheduling.

Complete $O(n^3)$ accounting

There are $O(n^2)$ relabels. A phase lies between consecutive relabels. Without relabeling, the outer cursor advances through at most n list vertices; a relabeling discharge starts a new phase. Thus there are $O(n^3)$ discharge calls. With valid doubly linked handles, move-front costs $O(1)$.

Between its own relabels, a current pointer scans its neighbors once. Each vertex relabels $O(n)$ times, so advances and relabel minimum scans total $O(n\Sigma\deg(u))=O(nm)$. Saturating pushes total $O(nm)$. A nonsaturating push empties u and ends discharge, so at most one occurs per call, for $O(n^3)$.

Total $O(n^3+nm)=O(n^3)$, since the simple capacity network has $m\leq n(n-1)$. Sparse adjacency and reverse records, heights, excesses, current pointers, and list need $O(n+m)$ space. These are arithmetic counts; capacity arithmetic and comparisons also pay their actual bit costs.

Choosing a generic operation is not free

Scanning all vertices for an active one and all neighbors at every step does not realize the $O(n^2m)$ basic-operation bound automatically. Maintain an active set instead: push changes only two excesses, and handles plus membership marks insert or remove in $O(1)$.

Use the same current rule for neighbors. All pointer advances and relabel scans total $O(nm)$, with $O(1)$ per push, realizing $O(n^2m)$ arithmetic time after initialization. Relabel-to-front uses a fixed all-vertex list and discharges to get $O(n^3)$; it is not merely another name for the active set.

Integer Ford–Fulkerson depends on F^*, Edmonds–Karp on nm^2, generic push-relabel on n^2m, and relabel-to-front on n^3. These arise from different worst-case mechanisms and do not directly predict measured machine speed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 26; § 26.5.

Algorithm_wjungle, handwritten notes on flow networks, escape routes, and vertex capacities, physical PDF pages 37–42.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study